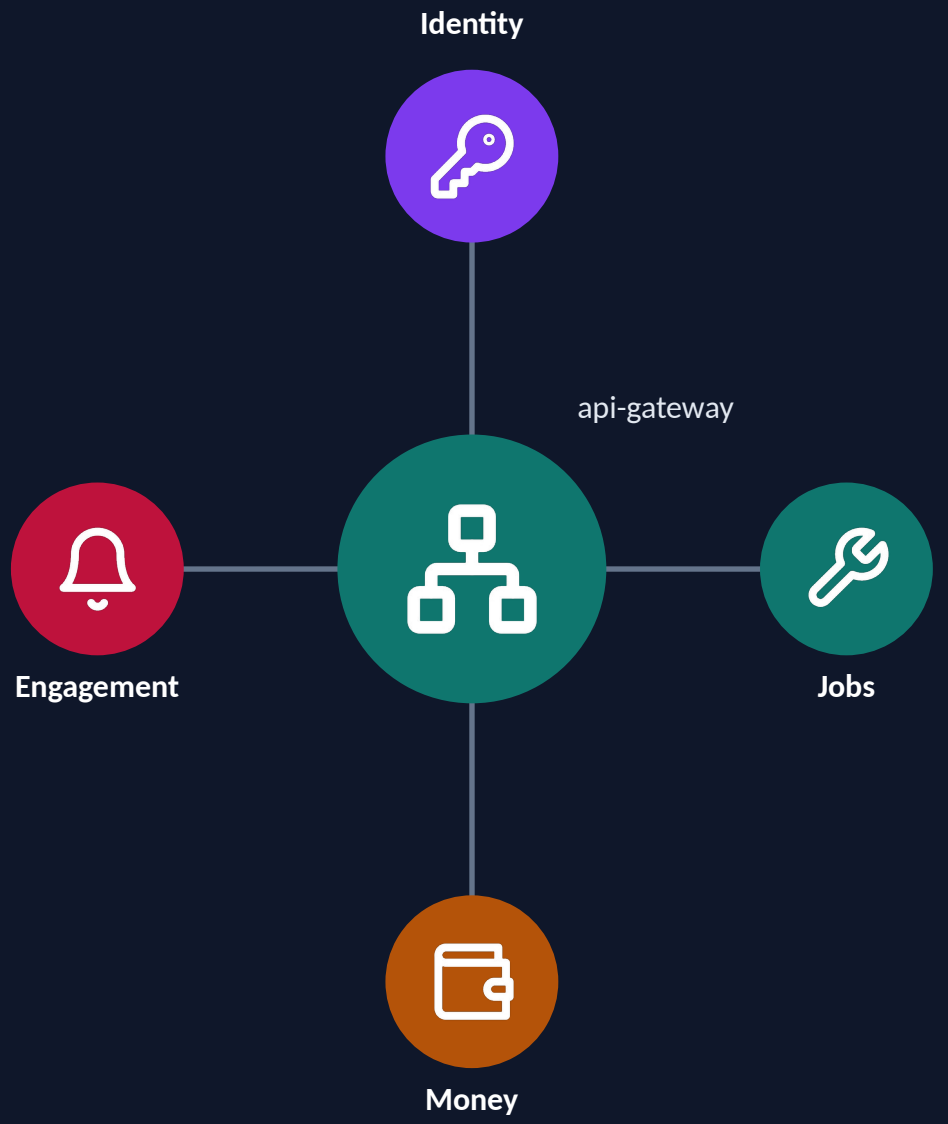
ON-DEMAND HOME SERVICES MARKETPLACE

# Ji-Hajoor

Platform architecture and system design

Architecture walkthrough · October 2026



# What changed on 30 October 2026

Security review, the reviews engine and naming: what this deck does not show yet

## Security review

- Web sessions: refresh token in an HttpOnly, Secure, SameSite=Strict cookie
- Passwords: 10+ characters, letter and digit; common, sequence and personal ones refused
- Rate limits keyed on Railway’s X-Real-IP; https-only image links; masked query strings in logs
- npm audit: 0 vulnerabilities in all three apps

## Reviews engine

- Review prompt after payment, one reminder after 24 h (ReviewReminder: 14 event types now)
- Public page per verified worker: /public/workers/{id} and its reviews
- Workers reply to reviews; share-my-profile card in the partner app
- Booking-flow CI now covers prompt, moderation, public page and reply

## Names used everywhere

- Apps: Ji-Hajoor (customers), Ji-Hajoor Pro (workers), Ji-Hajoor Admin (staff, My agency)
- People: customer, worker (Provider in code), agency (Tenant in code)
- Agencies invite workers; agency share 0–50% of the worker’s net

## Read next

- JiHajoor_Tech_Stack.pptx: current stack and security state
- JiHajoor_Pitch_Deck, Revenue_Model, Brand_and_Marketing
- User guides: docs/guides/; PDFs in docs/pdf/
- Security findings later in this deck show their state on 18 October

# Ji-Hajoor at a glance

Customers book verified professionals; agencies and staff keep the marketplace running

### Customer

Books scheduled or emergency jobs, tracks the pro live, chats, pays and reviews

### Service provider

Verified by documents and a background check; accepts offers and runs the job

### Agency (Tenant)

Covers an area with its own team; takes bookings matching could not fill

### Platform staff

Admin, finance, dispatch and support roles: verification, refunds, pricing, complaints

**20**
backend services

**5**
deployables (JVMs)

**3**
web apps, 2 also on Android and iOS

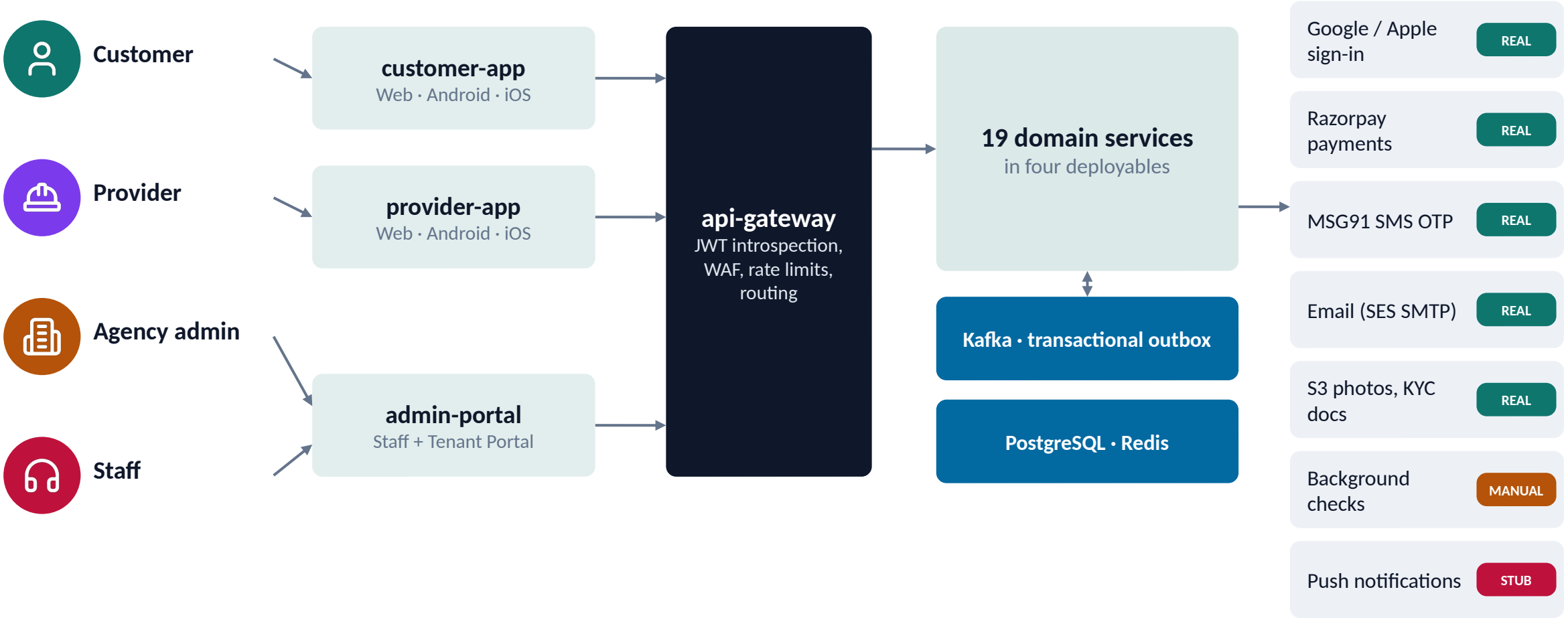
**13**
Kafka event topics

**19**
booking states

**45**
versioned Flyway migrations

# Who and what the platform talks to

Every client call enters through one gateway; services talk to each other over HTTP and Kafka




*Photos and verification documents go to S3 when a bucket is set and reachable, local disk otherwise; background checks are recorded by an admin.*

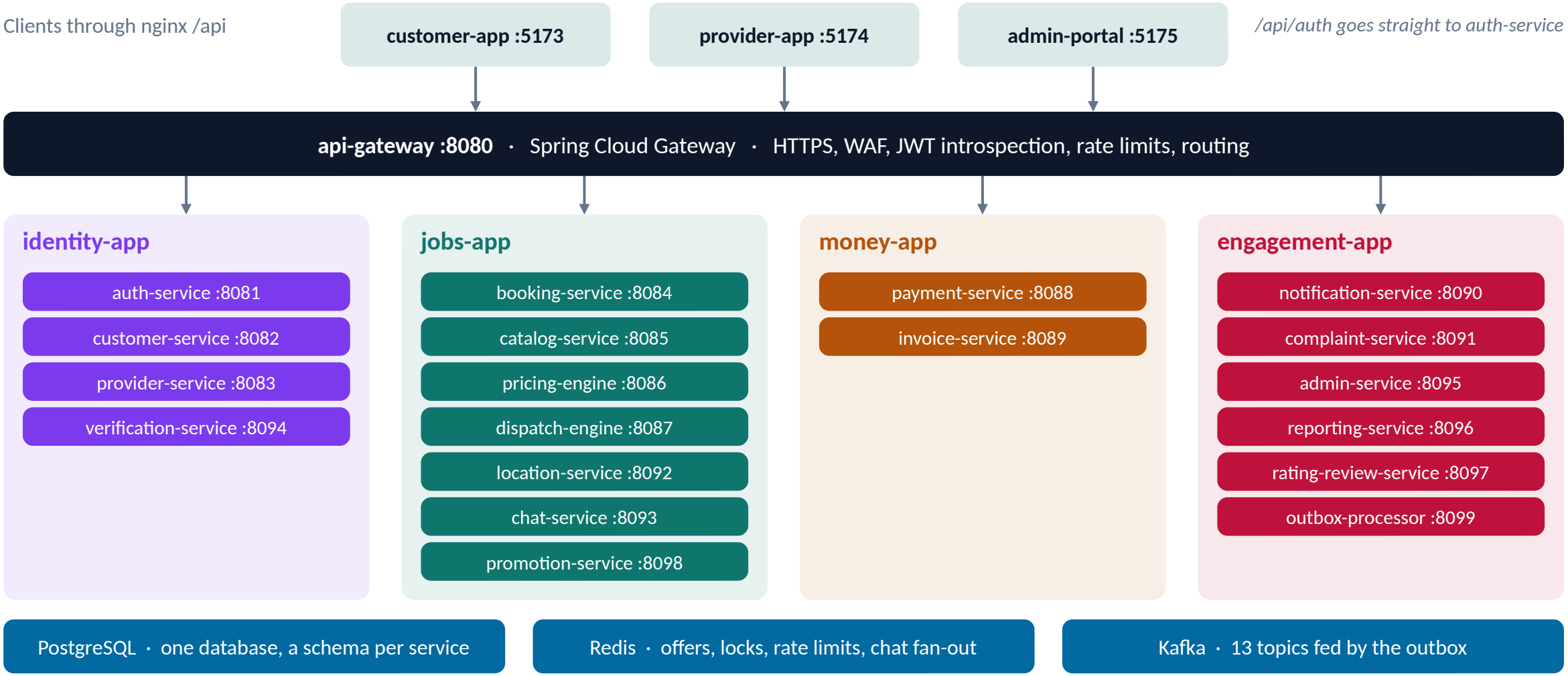
01

# How the system is put together

Deployables, the request path and the service host

# Twenty services in five deployables

Each deployable is one JVM and one image; every service keeps its own port, config and schema



# One JVM, many isolated services

shared/jihajoor-service-host starts each hosted service as its own Spring application context

**jobs-app · one JVM · one container image**

**booking-service :8084**
own context · booking-service.yml · booking schema

**catalog-service :8085**
own context · catalog-service.yml · catalog schema

**pricing-engine :8086**
own context · pricing-engine.yml · pricing schema

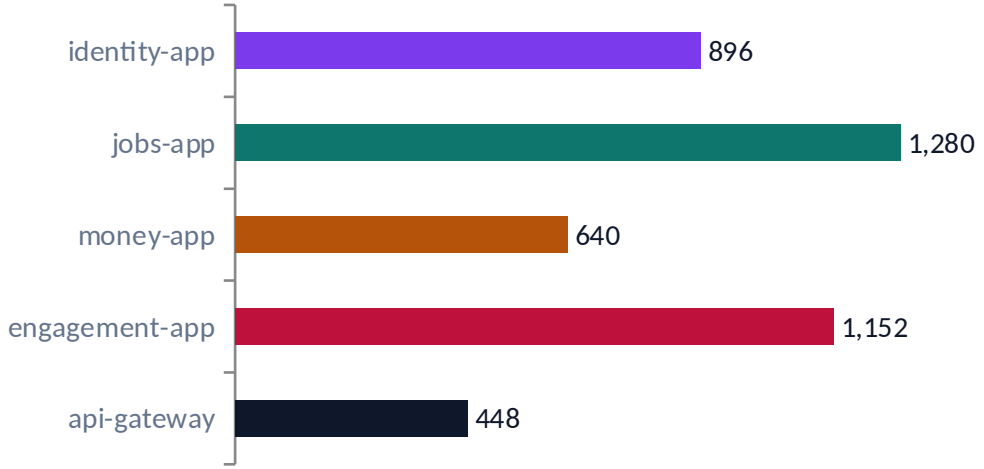
**dispatch-engine :8087**
own context · dispatch-engine.yml · dispatch schema

**location-service :8092**
own context · location-service.yml · location schema

**chat-service :8093**
own context · chat-service.yml · chat schema

**promotion-service :8098**
own context · promotion-service.yml · promotion schema

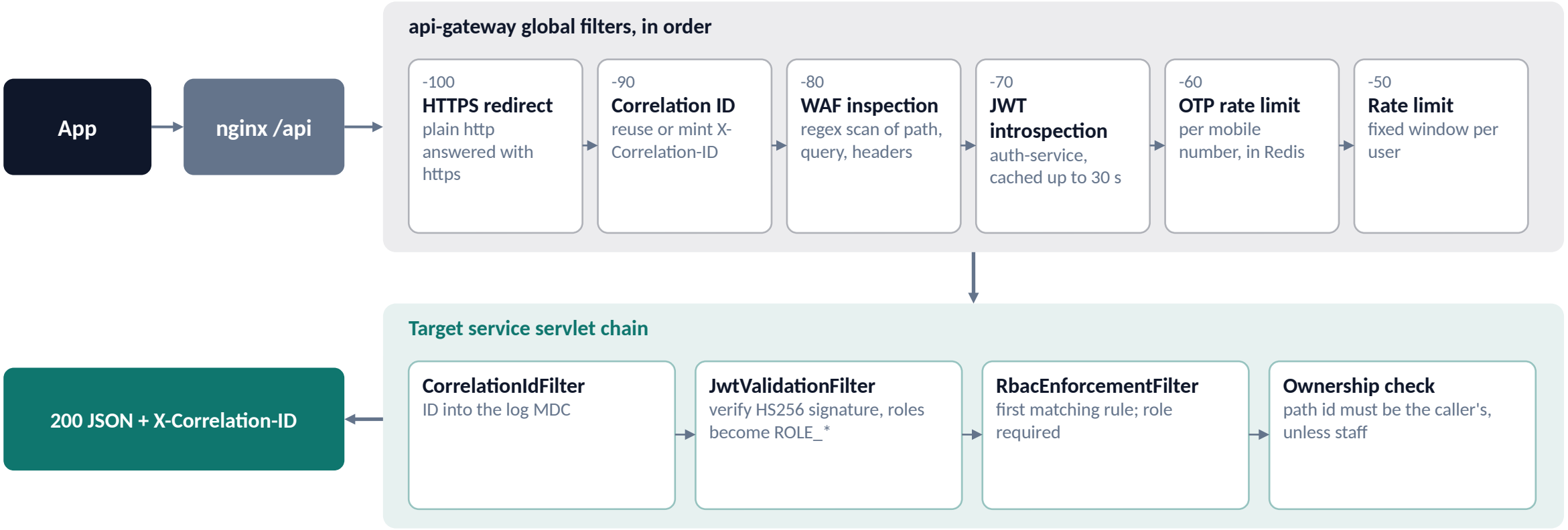
Container memory limit, local Compose (MB)




- Movable: a service can move to its own deployable without code changes
- Per-service settings: a PRICING_ENGINE__ prefix reaches one service only
- Health: the container probe checks every hosted service's readiness

# What happens on every authenticated request

Two filter chains: reactive in the gateway, then servlet inside the target service




Introspection results are reused for at most 30 s and never past the token's exp; an auth-service outage still denies

Rules match the decoded path, case-insensitively; the firewall answers ; %2F // and /../ with 400 first

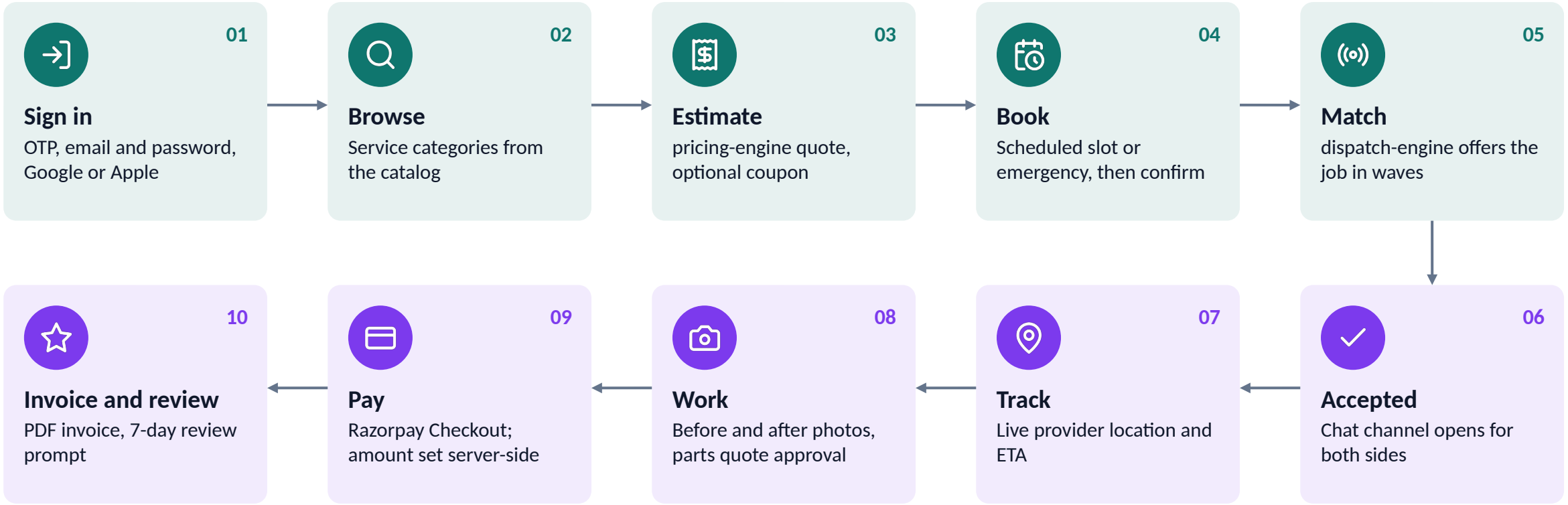
02

# From a tap on Book to an invoice

Journey, dispatch, lifecycle, payment and events

# The customer journey end to end

Ten steps, with two safety nets: agency fallback when nobody accepts, and complaints when a job goes wrong

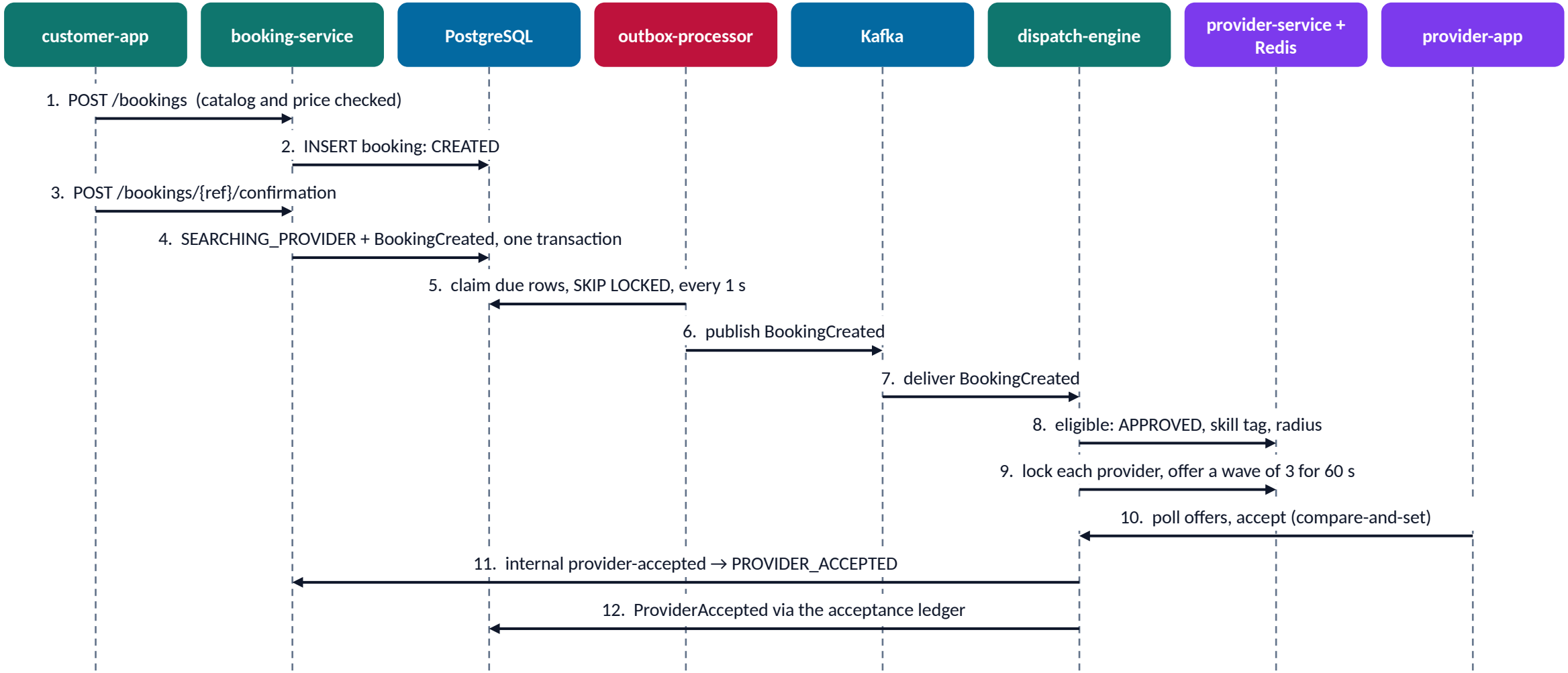



**Nobody accepts in 5 min** → covering agencies' queue (AWAITING_ASSIGNMENT), otherwise SEARCHING_FAILED

**Unhappy with the job** → complaint with an SLA clock, escalation, refund decision

# Booking to acceptance, step by step

The booking change and its event commit together; dispatch picks the event up from Kafka



# How dispatch picks a provider

Score everyone eligible, offer the best few at once, widen the search, then hand off to agencies

1. BookingCreated consumed
2. Resolve address coordinates and skill tags
3. Fetch eligible providers within the radius ↔ nobody? widen radius
4. Score and rank the candidates
5. Offer a wave: lock each in Redis, 60 s window ↔ all decline: next wave
6. **First accept wins; other offers withdrawn**

**Radius used up or 5 min passed** → searching-failed: agency queue or SEARCHING_FAILED

## Scoring signals

**Distance**
default weight 0.30

**Availability**
default weight 0.25

**Rating**
default weight 0.20

**Skill**
default weight 0.15

**Performance**
default weight 0.10

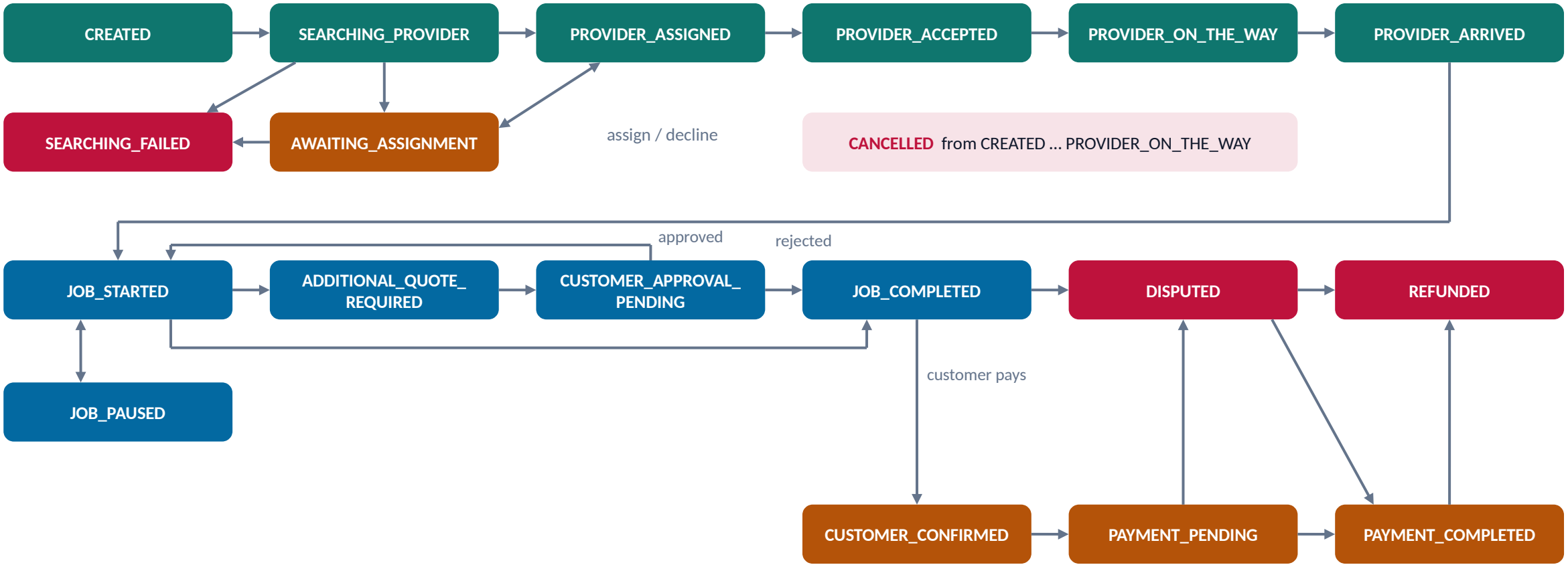
Weights must sum to 1. All settings are saved in the database and tuned in the Admin Portal

| Tunable | Range / default |
|---|---|
| Offer wave size | 1–10, default 3 |
| Offer window | 60 s, never past the hand-off |
| Agency hand-off | 1–30 min, default 5 |

*Saved in the database: survives restarts, every replica within 10 s*

# Booking lifecycle: 19 states

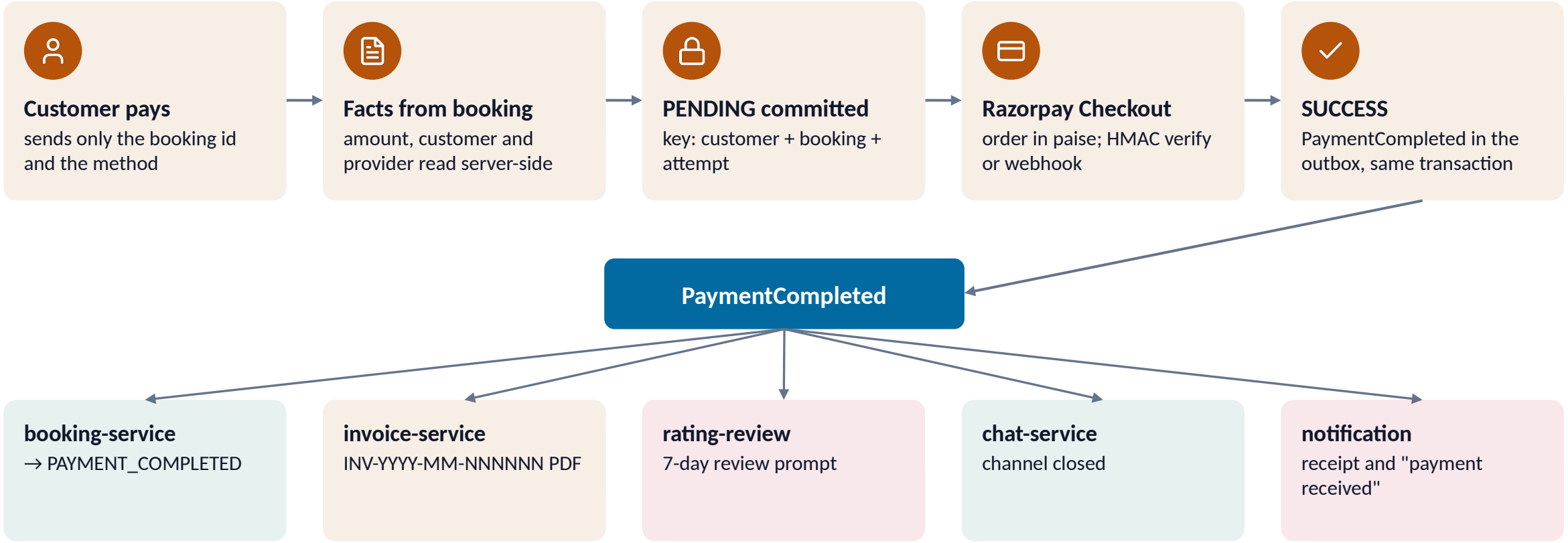
BookingStateMachine is the single source of truth; an illegal transition is refused with 409




*Passed-through states are entered and left in one transaction: audited, but no event*

# Payment, invoice and what follows

The PENDING row commits before the gateway is called, so a charge never happens without a record




The provider's wallet credit is sent at once; WalletCreditSweeper re-sends an unconfirmed one every minute, and provider-service applies a booking's credit only once.

# Reliable events: the transactional outbox

A domain change and its event share one transaction; delivery is at-least-once, handling is once

**Producer**
Domain row and outbox_event in one transaction. Propagation.MANDATORY: no event without one.

→

**outbox-processor**
Every 1 s, batch 100. Claims rows with FOR UPDATE SKIP LOCKED and a 2-minute lease, so relays never share a row.

→

**Kafka**
acks=all, idempotent producer. Every record carries an eventId header.

→

**Consumer**
Checks processed_event first; handler and dedupe row share a transaction. 3 failures go to topic.DLT.

**Publish fails** → Error stored, next_attempt_at = now + backoff (1 s → 60 s) → **10th attempt: FAILED, manual replay**

**Duplicates are expected, not harmful.** A crash after the broker ack or an outlived lease can publish twice; consumers dedupe on eventId, so each event takes effect once.

# Who listens to which event

13 topics from 5 producers; notification-service listens to all of them

| Topic | Produced by | booking | dispatch | invoice | chat | rating | location | notify |
|---|---|---|---|---|---|---|---|---|
| **BookingCreated** | booking | | ● | | | | | ● |
| **ProviderAssigned** | booking | | | | | | | ● |
| **ProviderAccepted** | dispatch, booking | | | | ● | | | ● |
| **ProviderRejected** | dispatch | | | | | | | ● |
| **ProviderArriving** | booking | | | | | | | ● |
| **ProviderArrived** | booking | | | | | | | ● |
| **JobStarted** | booking | | | | | | ● | ● |
| **JobCompleted** | booking | | | | | | | ● |
| **PaymentCompleted** | payment | ● | | ● | ● | ● | | ● |
| **BookingCancelled** | booking | | ● | | ● | | | ● |
| **ReviewSubmitted** | rating-review | | | | | | | ● |
| **ComplaintCreated** | complaint | | | | | | | ● |
| **ComplaintStatusChanged** | complaint | | | | | | | ● |

*Provider ratings travel over HTTP instead: rating-review-service pushes each provider's aggregate to provider-service after every review change, retried until it lands.*

03

# Platform foundations

Real-time, agencies, security and data

# Real-time: live location and chat

Browsers cannot set headers on SSE or WebSocket, so the gateway accepts ?access_token= on these two paths only

## Live location

**provider-app posts each GPS fix**
POST /locations/{bookingId}, at most one per 5 s

↓

**location-service**
latest position in Redis, trail in location_history

↓

**customer-app follows along**
snapshot, then the SSE stream; ETA estimated in the app

*Reads are ownership-checked; JobStarted ends the feed.*

## Chat (repaired in this review)

**Both apps open a WebSocket**
/chat/channels/{bookingId}/ws?access_token=…

↓

**api-gateway**
moves the token into the Bearer header

↓

**chat-service handshake**
booking participants only: 401, 403 or 404 otherwise

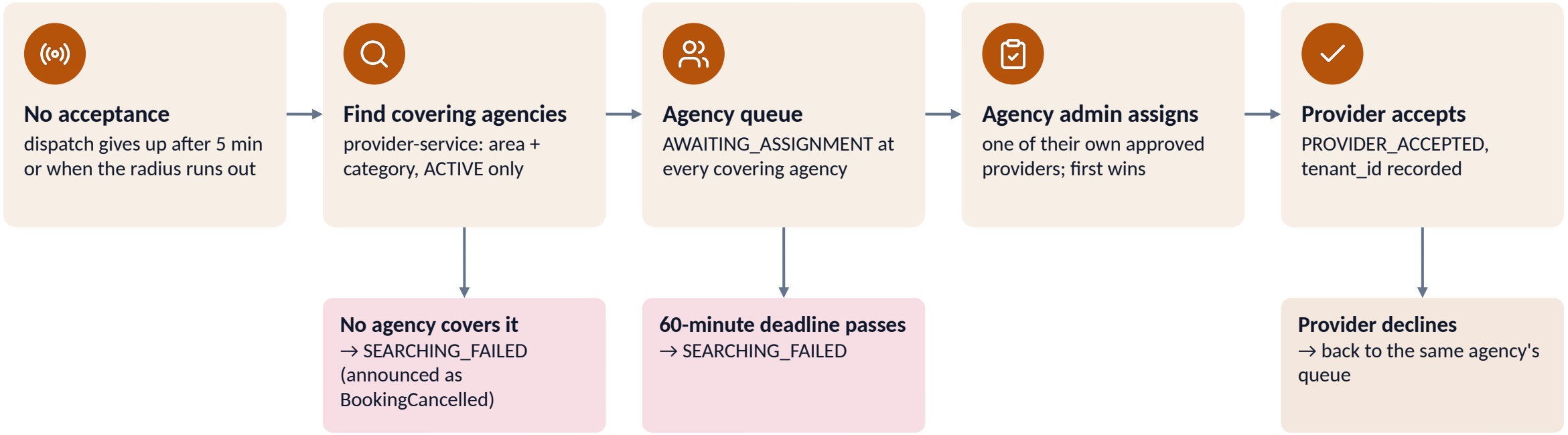
↓

**ChatService, then Redis**
masks numbers, stores, publishes once; every node delivers

*Works across instances: Redis pub/sub fan-out, presence in Redis with a 10 s TTL.*

# When nobody accepts: agency fallback

Agencies (Tenants) cover an area and a set of categories with their own team of providers




**Tenant Portal in the Admin Portal:** Requests (live badge, polled every 15 s), Team and Jobs. Customers, catalog and pricing stay shared; earnings still go to the provider.

# Security in four layers

Defence in depth from the edge to the data

## Edge

**HTTPS redirect**
plain http answered with https

**WAF inspection**
regex rules over path, query and headers

**Rate limits**
per user, and per number for OTP, in Redis

## Identity

**OTP sign-in**
5 wrong codes lock the number for 30 min

**Passwords and OIDC**
bcrypt; Google and Apple tokens checked by audience

**Token rotation**
short-lived access JWT; refresh replay is detected

## Authorization

**RBAC rule maps**
18 services, first matching rule wins

**Ownership checks**
path id must be the caller's, unless staff

**Internal API key**
/internal/** for service-to-service calls

## Data

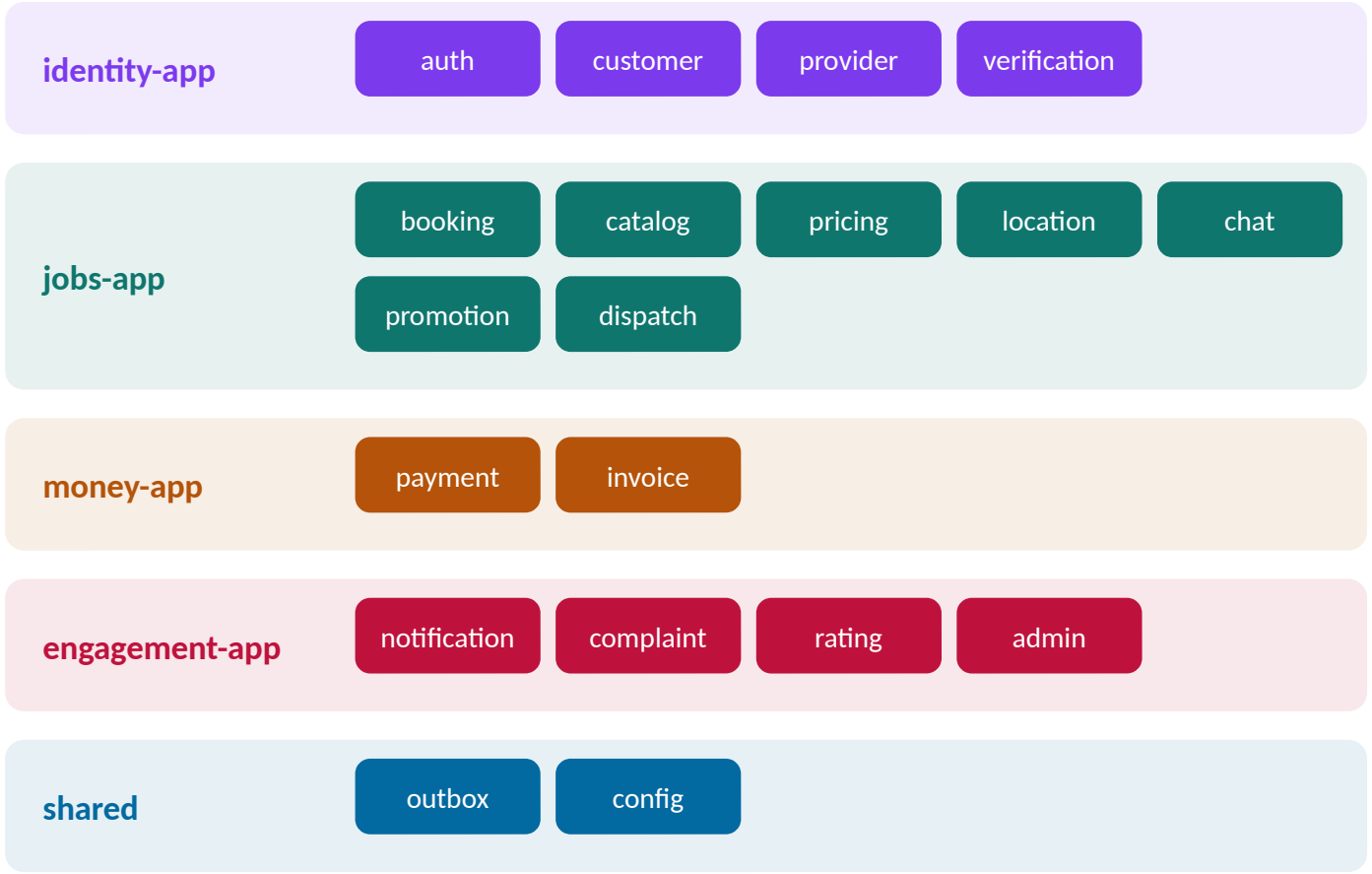
**Schema per service**
no service reads another's schema

**PII protection**
encrypted fields, masked log output

**Chat masking**
phone numbers removed before storage

# Data: one database, a schema per service

Flyway migrates each schema at startup; Hibernate only validates




66 Flyway migrations; each V1 baseline is frozen, later changes are incremental

No cross-schema foreign keys: references are UUIDs held by value

Redis: dispatch offers and locks, rate limits, OTP codes, location cache, chat fan-out and presence

config.app_config rows hold platform settings; environment variables still win

*dispatch keeps only its settings row; offers and locks live in Redis*

04

# Delivery and status

Where it runs, how it ships, and what is still open

# Three ways to run the same five images

Docker Compose for development, Railway for the live environment, Terraform and Helm for AWS

## Local

Docker Compose on a developer machine

- 3 nginx web apps :5173–5175
- 5 Java containers, 20 services
- Host PostgreSQL 18, Kafka 4.1, Redis
- Dev mail log or Mailpit; photos on disk or S3

## Railway (live)

Deployed from the main branch

- 3 web apps behind the edge, https enforced
- 5 backend services from one Dockerfile (APP=…)
- Postgres, Redis and Kafka services
- SMTP and S3 settings as variables

## AWS target

Terraform + one Helm chart

- CloudFront and WAF → API Gateway → ingress
- EKS: 5 Helm releases of one chart
- RDS Multi-AZ, MSK (TLS + SCRAM), ElastiCache
- S3 job photos, SES email, Secrets Manager

# Delivery pipeline

Nine GitHub Actions workflows: checks on every pull request, images and staging on every push to main

**Pull request**

- **PR checks** — lint, test, coverage, image build

**Push to main**

- **CI** — build, test, image, deploy to staging → **Build services reusable**; **Lint delivery workflows, chart, ports**; **Deploy services Helm, reusable**
- **Frontend CI** — format, lint, type-check, build, image
- **Booking flow** — full booking through the Railway layout
- **Railway smoke** — auth and gateway unit tests

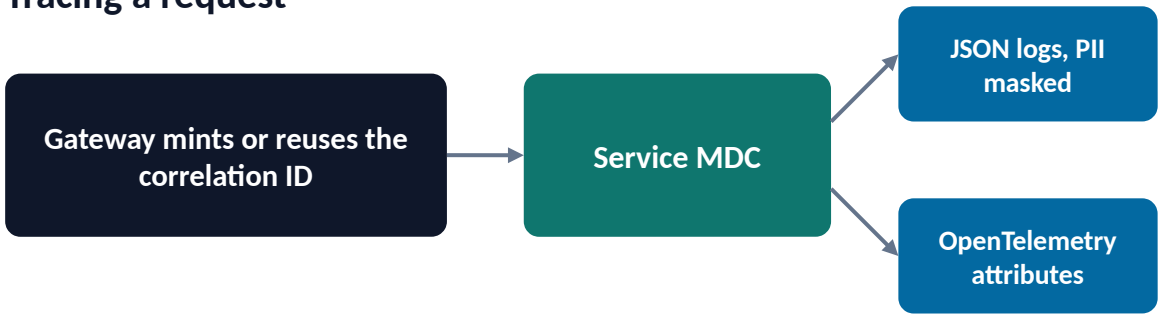
**Manual**

- **CD production** — promote CI images by commit SHA → **Deploy services → EKS via Helm**

# Observability and resilience

One correlation ID follows a request everywhere; outbound calls are bounded and retried

## Tracing a request




Health on every service's own port, probed by Compose and Helm

- /health/liveness
- /health/readiness
- /prometheus
- /metrics

## ResilientCall on synchronous calls

**Timeout**
bounded per call

**Retry × 3**
exponential backoff

**Circuit breaker**
opens at 50 % failures over 10 calls, for 30 s

Used by booking, dispatch, payment and provider clients. This review added a 3 s timeout to customer-service's call to booking-service, the one client without one.

# What is real and what is still a stand-in

Integrations checked in source during this review

## Real integrations

- **Google and Apple sign-in**
  ID tokens validated against the client id
- **Razorpay, the only gateway**
  orders, Checkout, verify, webhook, refunds
- **MSG91 SMS**
  sign-in OTPs; OTP_FIXED_CODE is a test mode
- **Email over SMTP**
  Amazon SES in AWS, Mailpit locally
- **S3 photos and KYC documents**
  when a bucket is set and reachable; disk otherwise
- **Refunds and ratings**
  complaint refunds via payment-service; reviews set ratings

## Stand-ins

- **Push notifications**
  no device tokens are stored
- **Background checks**
  an admin records the result
- **Invoice PDF storage**
  kept in memory; re-rendered from the row on download
- **Admin dashboard metrics**
  a stub source reports zeros
- **Provider payouts**
  settlements are simulated (no RazorpayX)

# Review status: what this review fixed

Every earlier finding re-checked, the live chat defect repaired, and all eight open items implemented

**20 / 20**

findings from the 3 October review verified fixed in today's source

**8 / 8**

open items implemented on 18 October, each unit-tested and started against PostgreSQL

## Fixed in this review

- **Chat works end to end** channel lookup, WebSocket, STOMP bypass closed, provider chat screen
- **Chat across instances** Redis pub/sub fan-out; presence in Redis with a 10 s TTL
- **Dispatch settings in the database** five weights and six tunables; every replica within 10 s
- **Complaint refunds move money** through payment-service and Razorpay, idempotent
- **Reviews set provider ratings** pushed after each change, retried until applied
- **Email over SMTP** Amazon SES in AWS, Mailpit locally; codes kept out of logs
- **Job photos to S3** when configured and reachable; local disk otherwise
- **One Kubernetes namespace** IRSA per deployable, KEDA in the chart, label-based monitoring
- **Stripe removed** Razorpay is the gateway; no Stripe secret needed

05

# What changed on 18 October

The eight open items implemented, and the files removed along the way

# What was removed, and why

Every deletion replaced something dead, misleading or unused; each can be restored with git restore

| Removed | Why | Replaced by |
|---|---|---|
| **infra/k8s/observability/hpa/ (8 HPA files: auth, booking, customer, dispatch, invoice, location, payment, provider)** | Each scaled a per-service Deployment (auth-service, booking-service, …) in one of six per-domain namespaces. The platform deploys 5 deployables into one namespace, so they pointed at nothing; a second HPA on a real Deployment would fight the chart's. | The chart's HPA, one per deployable, in the release namespace (helm/jihajoor-service/templates/hpa.yaml) |
| **infra/k8s/observability/keda-scaledobjects.yaml** | Same dead targets and namespaces. Watched topics and groups the code never uses (booking.events, dispatch-engine; real: BookingCreated, dispatch-engine.booking-created). Hard-coded MSK hosts and a committed Secret with PLACEHOLDER credentials (review: High). | Chart template keda-scaledobject.yaml with the real consumer groups (keda.enabled), plus keda-kafka-trigger-auth.yaml applied into the namespace |
| **payment-service StripeGatewayAdapter.java** | Razorpay is the gateway. The adapter only simulated charges, yet clients could choose it, and payment-service would not start without STRIPE_WEBHOOK_SECRET. | Nothing: Razorpay only. The secret left Compose, Helm, CI and .env.example; tests use a test-only second gateway |
| **dispatch-engine MatchingWeightsStore.java (and its test)** | Kept the matching weights in memory: an admin change was lost on restart and differed on every replica. | JpaDispatchSettingsStore over the dispatch.dispatch_settings table (V3) |

*They show as staged deletions because they were removed with git rm. To bring one back: git restore --staged <file> && git restore <file>*

# Kubernetes: six namespaces became one

The manifests now match what the pipeline actually deploys: five releases into K8S_NAMESPACE

## Before

**auth-customer-provider**
HPA: auth, customer, provider

**booking-dispatch-location**
HPA: booking, dispatch, location; KEDA

**payment-invoice-rating**
HPA: payment, invoice; KEDA

**notification-chat-complaint**
KEDA: notification, chat

**admin-reporting-promotion**
nothing deployed here

**catalog-pricing-verification**
nothing deployed here

Targets named per service (auth-service, …) never exist; IRSA roles trusted service accounts nothing runs as

## After

**namespace jihajoor = the pipeline's K8S_NAMESPACE**

| | | |
|---|---|---|
| api-gateway | HPA | IRSA |
| identity-app | HPA | IRSA: S3, PII KMS |
| jobs-app | KEDA lag / HPA | IRSA: S3 |
| money-app | KEDA lag / HPA | IRSA: S3, PII KMS |
| engagement-app | KEDA lag / HPA | IRSA: S3 |

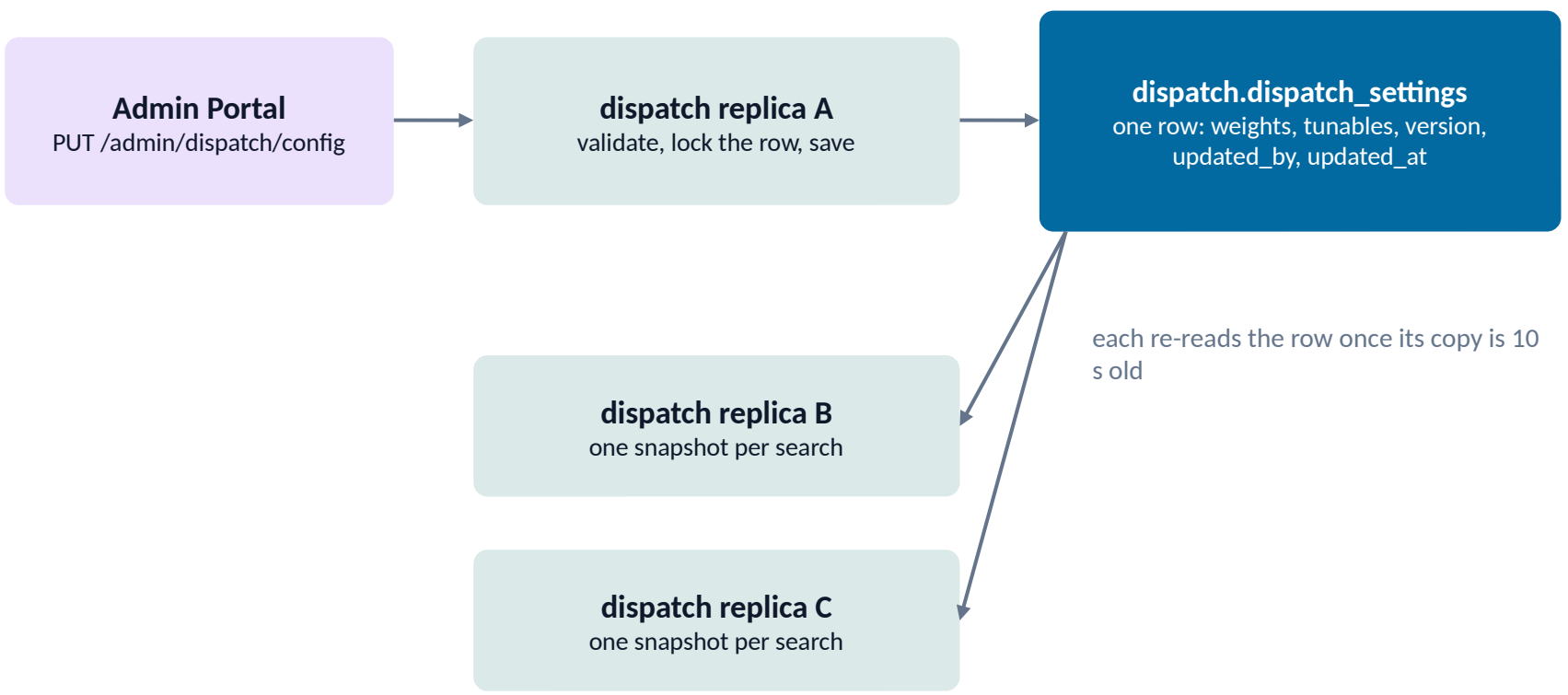
Prometheus scrapes every hosted service's port, by label

Fluent Bit: one CloudWatch log group per deployable

Terraform: k8s_namespace, one IRSA role per deployable

# Dispatch settings live in the database

One row holds the five weights and the six tunables; every replica reads it




**Until the first save**
no row: the DISPATCH_* defaults apply

**Concurrent saves**
last write wins under a row lock; a partial PUT never undoes another

**Database rules**
CHECK: each weight 0–1 and the five sum to 1

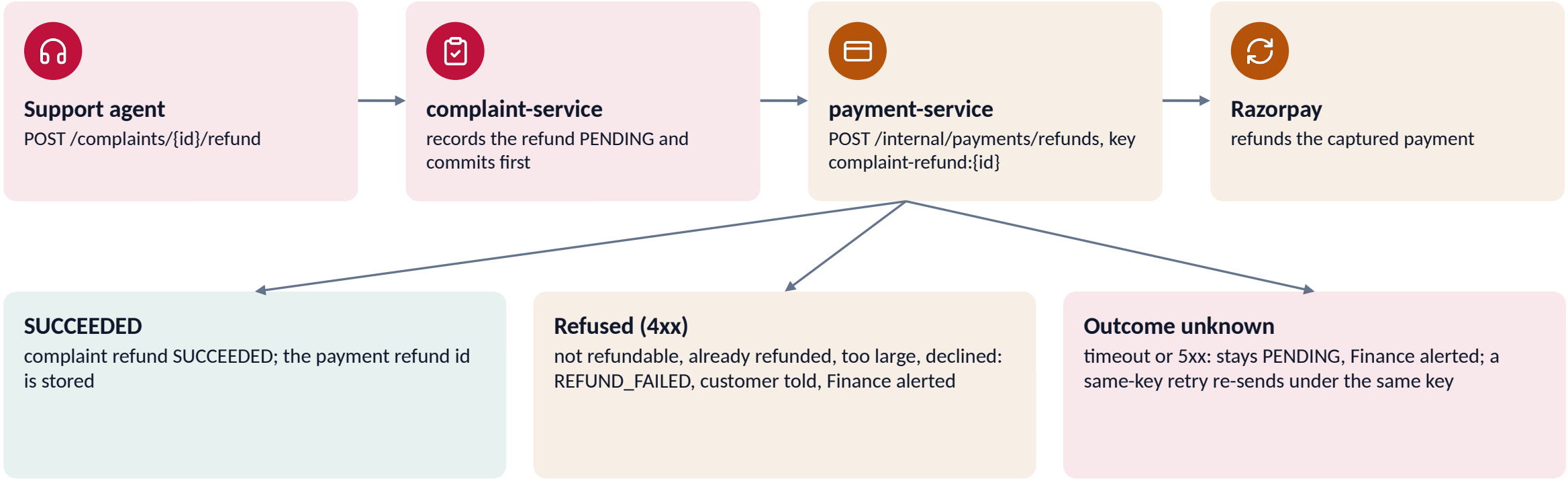
**If the DB is down**
replicas keep their last-read settings

**Default weights**

| distance 0.30 | availability 0.25 | rating 0.20 | skill 0.15 | performance 0.10 |
|---|---|---|---|---|

# Complaint refunds now move money

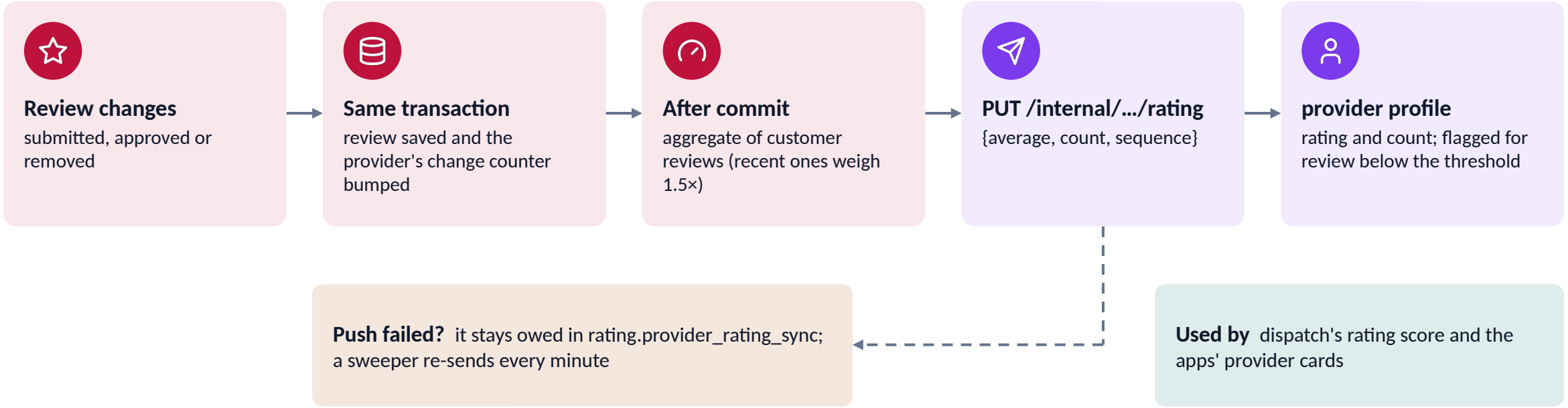
complaint-service asks payment-service, which refunds through Razorpay; a retry can never refund twice




A retry while an attempt is still in flight (under 5 min) gets 409 REFUND_IN_PROGRESS. COMPLAINT_PAYMENT_CLIENT=logging keeps the old stub for local use only.

# Reviews now set provider ratings

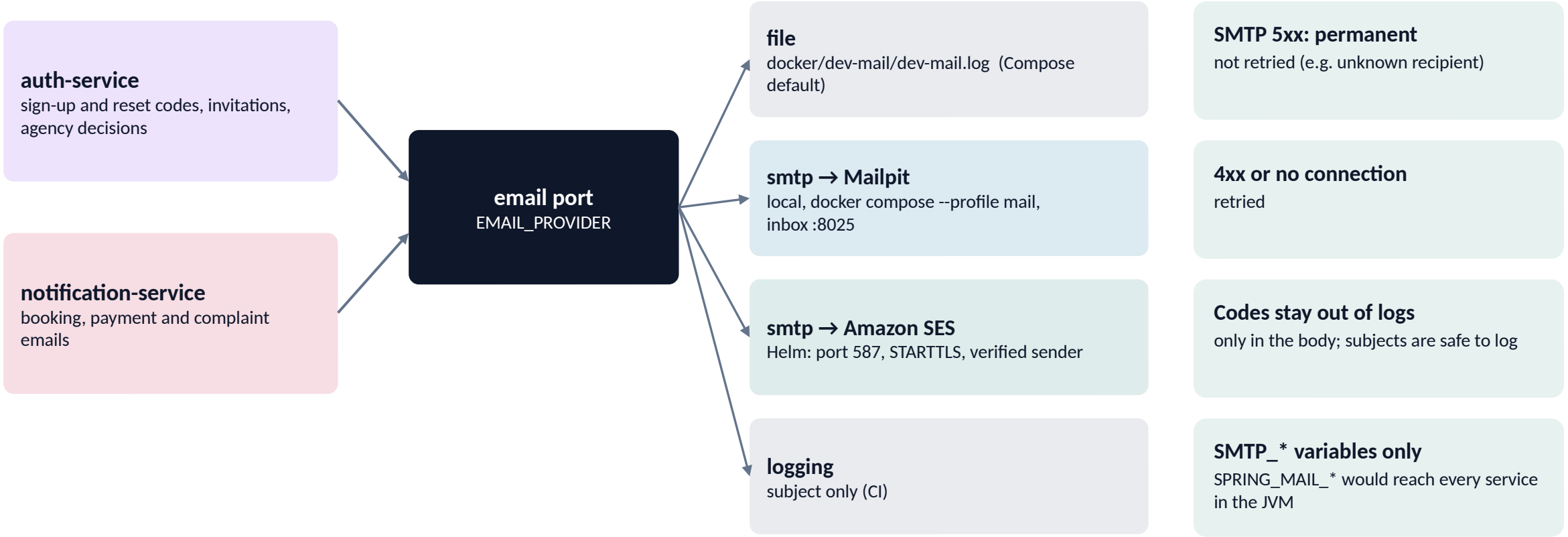
rating-review-service owns reviews and pushes each provider's aggregate; the push is an idempotent set




A push with an older sequence is ignored, so a late retry can never roll a rating back. Provider-to-customer reviews never count. Existing reviews are back-filled on the first sweep after deploy.

# Email over SMTP

Modelled on your shiprock notification service: Spring Mail, Amazon SES in AWS, Mailpit locally

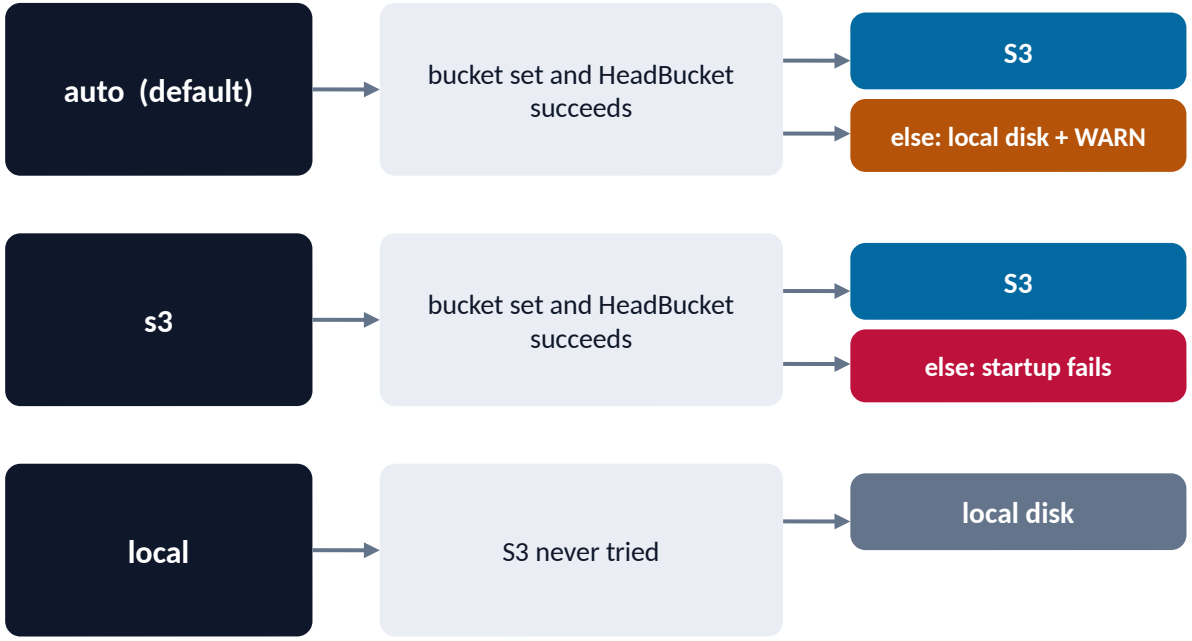



Resend now answers the same for every address, so it cannot reveal a pending sign-up.

# Job photos: S3 when available, local otherwise

Chosen once at startup from MEDIA_STORAGE_PROVIDER; each photo records where it went

## At startup




## Then, per photo

**Upload**
POST /bookings/{key}/photos → the chosen store; key bookings/{id}/{photoId}.jpg

**S3 fails mid-request**
503 MEDIA_STORAGE_UNAVAILABLE: never quietly written to one server's disk

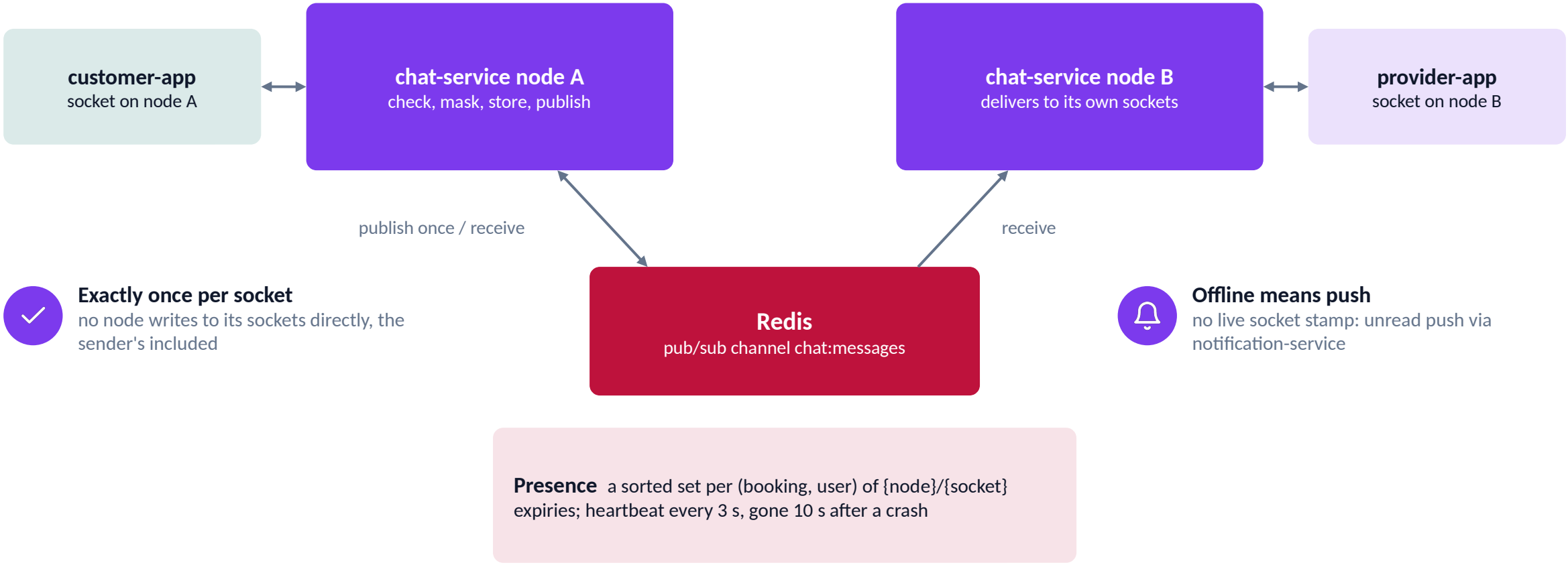
**Recorded**
job_media.storage = LOCAL or S3 (V5), so old photos stay readable

**Read**
GET /bookings/{key}/photos/{photoId}, streamed with the detail's access rule; shown in the provider app

*The old local adapter never wrote any bytes, so photos uploaded before 18 October answer 404 MEDIA_NOT_FOUND.*

# Chat across several instances

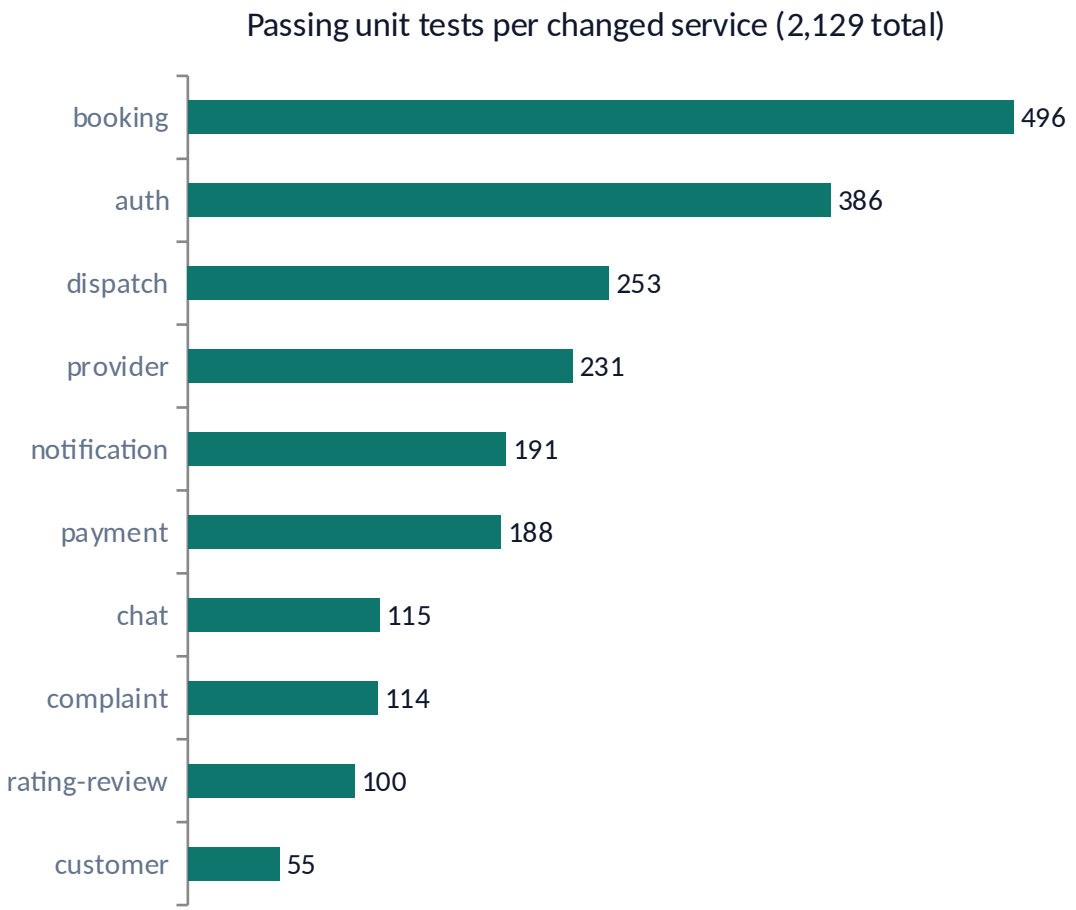
Each message is published once on Redis and every node delivers it to its own sockets




Redis down: the message is still stored and the recipient pushed. CHAT_CLUSTER=local keeps the single-instance mode.

# How the changes were verified

Unit tests per module, then every changed service started against a real PostgreSQL




### Migrations applied on a throwaway PostgreSQL 18

- booking V5
- complaint V3
- dispatch V3
- payment V4
- provider V5
- rating V2

Each changed service started from its packaged deployable; Hibernate schema validation passed in all of them, and payment-service started with no Stripe secret.

### Not verified yet

- ✕ the full stack end to end (Docker would not start here)
- ✕ real SES and S3; two chat nodes on one Redis
- ✕ terraform plan; provider-app unit tests (Node 24 needed)

06

# Load test and gap review

k6 through the gateway, then every app call checked against the services behind it

# How the load test drives the platform

Three populations at once; the booking journey crosses all five deployables and both Kafka hops

### browse 60 customers

catalog, price estimate, booking history, with 2–4 s of think time: the bulk of real traffic

### customers 12 customers

book a plumber, confirm, wait for a provider, watch the live location, pay when the job is done

### providers 8 providers

onboarded in setup, poll offers every 5 s, accept, share location, upload photos, complete

*Ramp 1 min, hold 4 min, ramp down; the k6 image runs in Docker on the stack's own network.*

## Pass criteria

| | |
|---|---|
| < 1 % | unexpected statuses |
| 500 ms | browse p95 (1.5 s p99) |
| 1.5 s | create and confirm a booking, p95 |
| 300 ms | provider location ping, p95 |
| 30 s | confirm → provider accepted, p95 |
| 15 s | payment → PAYMENT_COMPLETED, p95 |
| 0 | 429s from app-like traffic |

# Load profile: results

82 complete journeys and 12,487 requests, 4-minute hold on one laptop; 0.9 % unexpected statuses

**82**

complete journeys: booked, dispatched, done and paid

**26/s**

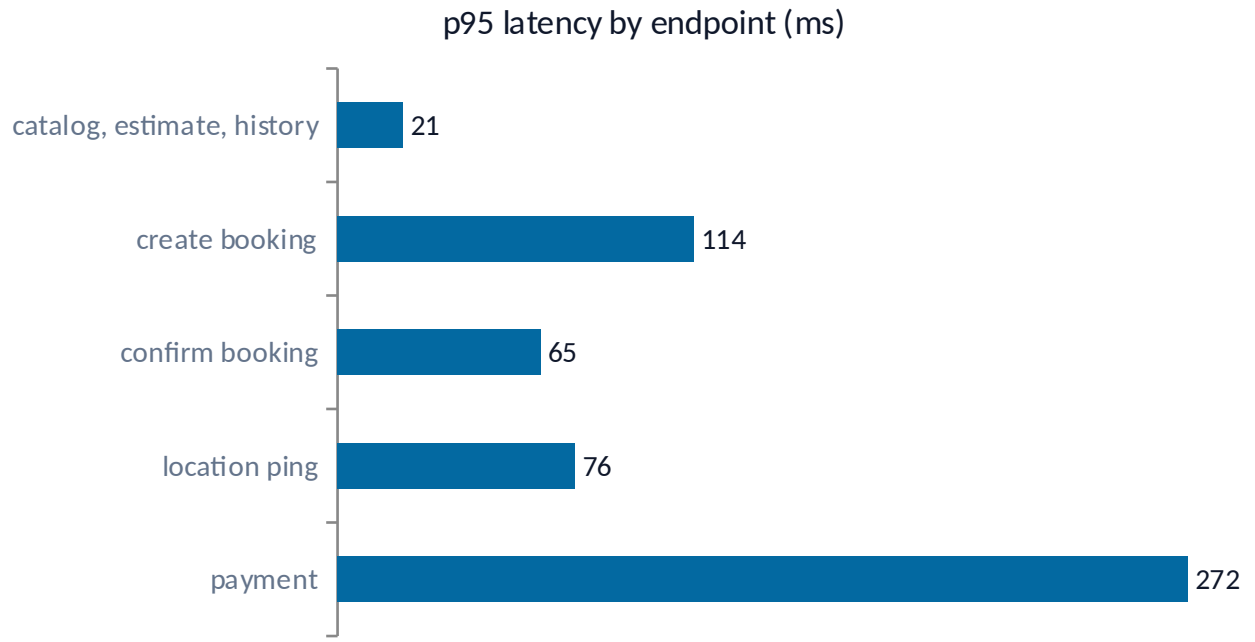
requests per second through the gateway

**0.9 %**

unexpected statuses

**21 ms**

browse p95 latency




## Asynchronous hops (wall clock)

**confirm → provider accepted**
p50 4.1 s · p95 13.7 s

**payment → PAYMENT_COMPLETED**
p50 1.0 s · p95 1.1 s

# What the load test showed

The services kept up; the limits that bit were policy and pool sizing, and two are fixed

### An idle provider costs a full offer window

with idle providers in range, confirm → accepted took 62 s at the median; with active ones 4.1 s. Dispatch waits the whole 60 s window per wave, and providers have no online/offline state.

### Notifications drained their connection pool

the shared consumer held a connection through every SMS, email and push send, with back-off, on eleven listeners against a pool of 4. Fixed: notification-service opts out of the wrapping transaction (0 pool timeouts in the re-run).

### Busy providers reach the 60-a-minute limit

25 refusals at minute boundaries for providers doing jobs back to back; the app uses about 40 a minute during a job. Now RATE_LIMIT_PROVIDER_PER_MINUTE; the value is a decision to make.

### Hot polls had no index

every provider poll scanned booking.booking by provider_id; skill tags, earnings, reviews, complaints and invoices too. Fixed: booking V6, provider V6, rating V3, complaint V4, invoice V2

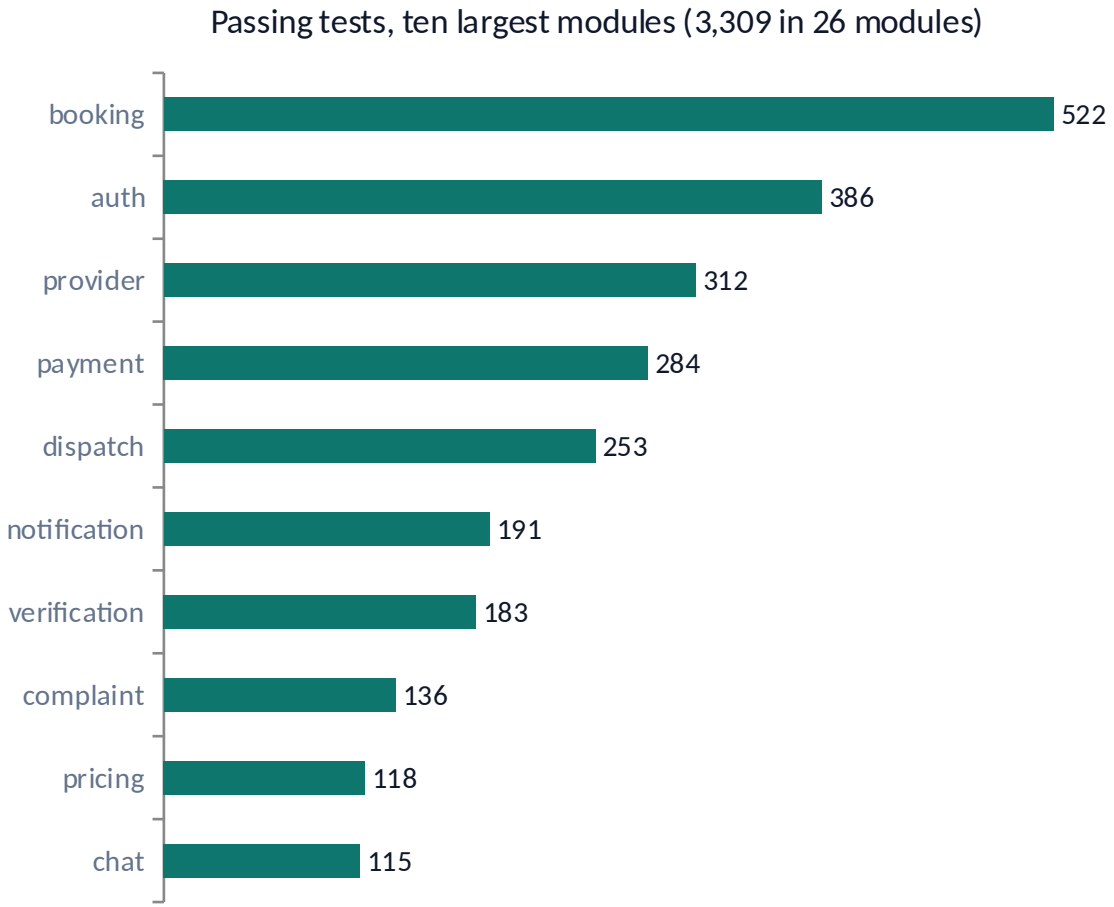
# Gaps found and fixed

Every call the three apps make was checked against the gateway, the controller, RBAC and the DTO

- **Verification documents stored**
  S3 or local disk; admins see them in the review dialog
- **Complaint refunds from the portal**
  Refund dialog, refund status, Re-send for an unknown outcome
- **Invoice download works**
  invoice by booking and PDF download; re-rendered when gone
- **Customers see job photos**
  before and after tiles on the booking detail
- **Saved addresses listed**
  GET /customers/{id}/addresses; the app no longer reuses deleted ones
- **Dead screen removed**
  Available Professionals called an endpoint that never existed
- **Chat reconnects safely**
  reloads history, refreshes an expired token, keeps undelivered text
- **Accept → job screen race**
  the provider app retries the first 404 while dispatch assigns
- **Provider status toggle**
  Suspend only offered to approved providers (no more 409s)
- **Logout through the gateway**
  the apps send their token, so the refresh token is revoked
- **SMTP misconfiguration retried**
  530 and 535 no longer drop the email as permanent
- **One rating threshold**
  REVIEW_RATING_THRESHOLD sets both services

# How it was verified

The whole reactor, the three apps, a fresh stack and the booking flow, then the load test




## New migrations applied on PostgreSQL 16

- booking V6
- provider V6
- rating V3
- complaint V4
- invoice V2
- verification V2

- ✓ booking flow end to end: 48 of 48 checks
- ✓ new endpoints via the gateway: 11 of 11
- ✓ three apps: tsc and eslint clean
- ✓ guardrails: 2,012 of 2,012 checks

## Not verified

- ✗ real S3 and SES; two replicas of jobs-app
- ✗ the apps in a browser; frontend unit tests
- ✗ terraform plan

# Still open: recommended next steps

What is left after this review, roughly by impact

01 **Rotate the leaked JWT secret**
every .env and environment still set to the old .env.example value

02 **Set the deploy prerequisites**
SES and SMTP, MEDIA_BUCKET, DOCUMENTS_BUCKET, IRSA role ARNs

03 **Provider online / offline**
idle providers hold an offer wave for the full 60 s window

04 **Size the provider rate limit**
busy providers reached 60 requests a minute under load

05 **Push notifications**
register device tokens in the apps and send them

06 **Refunds debit the provider**
a refund leaves the provider earning in place

07 **Store invoice PDFs**
in S3, like photos and documents; in memory today

08 **Real S3, SES, two chat nodes**
and terraform plan: not run in this review